

Advanced Cryptography

Elisabeth Oswald

Revision Lecture 2 — Side Channel Attacks, Security Notions, Efficient $(2, 2)$ Boolean Masking for AES, (d, d) Boolean Masking

ROADMAP

Side Channel Attacks

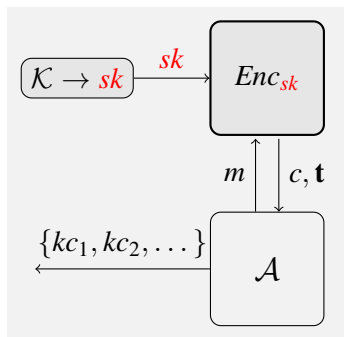
Attacks

Number of Needed Traces, SNR

Countermeasures

Masking, aka (d, d) Boolean Secret Sharing

KEY RECOVERY VIA SIDE CHANNELS



Most side channel attacks have key recovery as a goal: divide and conquer independent chunks kc_i of the secret key.

The adversary supplies some input (m) in this illustration, and they may receive output (c in this example) and a side channel “trace”.

Univariate side channels (i.e. there is a single measurement, like overall execution time)

Multivariate side channels (i.e. instantaneous power/EM).

SIDE CHANNELS

The “best” side channel is the one that you have access to:

- ▶ physical: via local access, typically for embedded systems; high quality **power/EM, timing** side channels
- ▶ logical: via an interface, either local or remote, the latter would be for cloud based systems; timing side channels, sometimes also power side channel

The “quality” of side channel information is quantified via the **signal-to-noise ratio (SNR)** (at time point t)

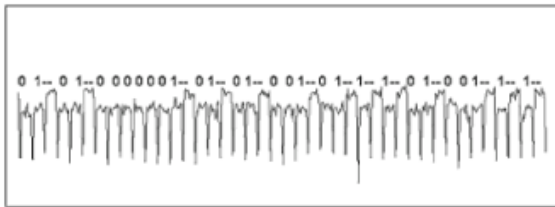
$$SNR(t) = \frac{\text{Var}(\text{Signal})(t)}{\text{Var}(\text{Noise}(t))} \quad (1)$$

To define “signal” and “noise” we typically must have “full knowledge” of an implementation (including being able to set/know internal secrets/randomness).

ATTACK STRATEGIES: SINGLE VS DIFFERENTIAL INPUT ATTACKS

Single input attack: the adversary has no key control but can supply an input. The input may be used in a “one-time encryption” fashion, i.e. the same key is never used twice. Examples for such cryptosystems: El Gamal, DSA, certain modes of operation, et.

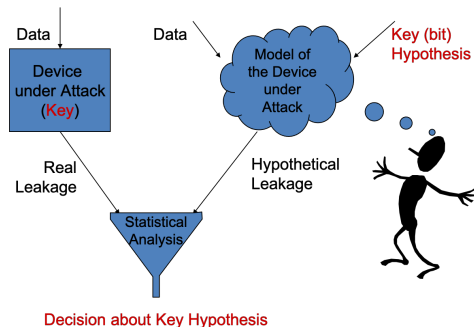
If the data dependency dominates what can be measured, then it is possible that a **single observation** of the action of a cryptographic process that uses a secret key reveals this key. Such an unfortunate situation can occur in practice, see an exemplary power trace of an RSA decryption below:



ATTACK STRATEGIES: SINGLE VS DIFFERENTIAL INPUT ATTACKS

Differential input attack: the adversary has no key control but can supply multiple/many different inputs.

I use the term “DPA style” or “differential style” even in the context of timing and EM attacks.

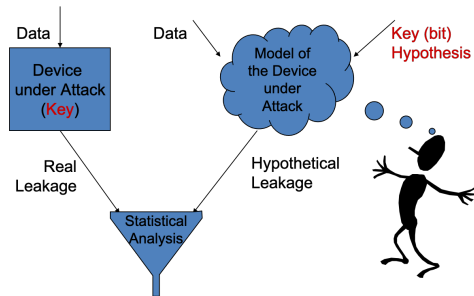


“Divide and conquer” principle: the adversary learns information about “chunks” of the secret key; chunks can be treated independent of each other; eventually the entire key is revealed. Whilst many factors are relevant for success (aka key recovery), the **number of needed traces** is the key criterion for the “quality of an attack”.

DIFFERENTIAL ATTACKS

There are a number of choices that are necessary of “configuring” a concrete attack:

- ▶ The model: intermediate value and leakage model
- ▶ The measurements: what side channel and how/what to measure
- ▶ The statistical analysis



In the notes I explain five distinctive steps/choices: when asked to explain a differential input attack, or, when asked to explain these five steps, then a correct answer will name all the five steps: choose intermediate for attack, record traces accordingly, compute hypothetical intermediate values using key guesses, compute hypothetical leakage values using leakage model, use distinguisher to get scores for key guesses.

EXERCISE 1, LAB SHEET 2

You were given a dataset called `WS1.h5`, which contains 200 traces plus the corresponding input bytes. You also got a very basic set of functions to perform various types of analyses, in particular, a correlation based attack, and a distance-of-means based attack (together with functions to predict power for `AddRoundKey` and `SubBytes`).

The exercise asks you to “play around” with the different configuration that there were in the supplied functions (the different configurations were just commented out):

What happens when targetting `AddRoundKey`?

What happens when changing to the LSB power model?

EXERCISE 1, LAB SHEET 2, CONT.

What happens when targetting `AddRoundKey`?

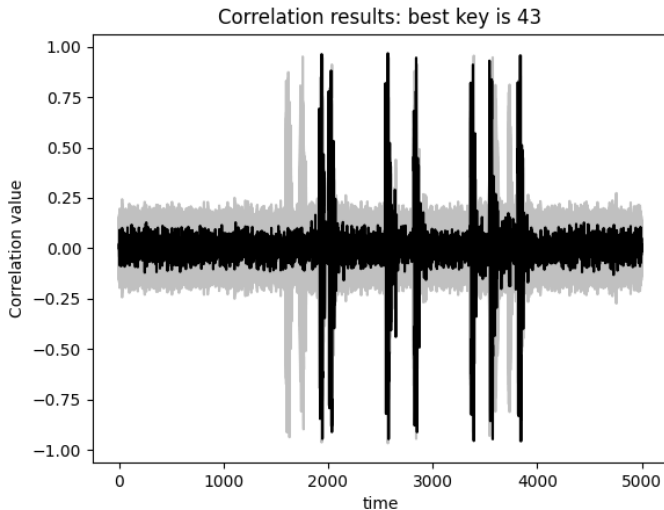
We actually looked at this during the lecture. The function `AddRoundKey` exclusive-ors a byte from the state and a byte from the key state. Suppose that the key is `0xFF` (it flips all bits from the state byte).

The correct key guess will correspond to the true key and all predictions will be the state byte with all bits flipped. Now consider a “close” incorrect key guess like `0xFE` (one bit is not flipped). Then the predictions corresponding to this key guess will be very “close” to the predictions of the correct key guess: in fact, each prediction will be at most one bit “away” from the correct value.

If the device leaks a function that has one component that is rather linear in the bits of the predictions, then the correct key guess, and the “close” incorrect key guesses will lead to very close predicted leaks. Ergo, their predictions will “correlate highly” with the measured traces: hence they lead to very close distinguishing scores.

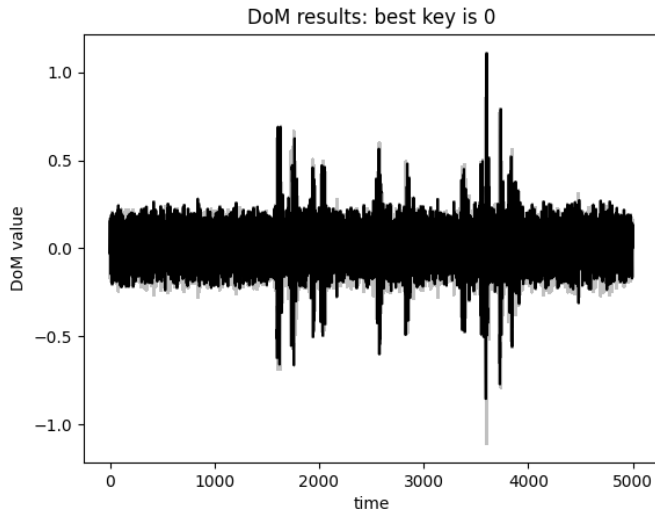
EXERCISE 1, LAB SHEET 2, CONT.

What happens when targeting AddRoundKey?



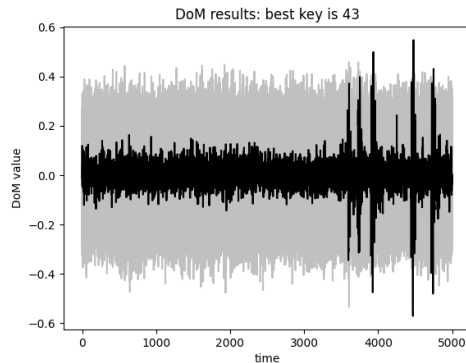
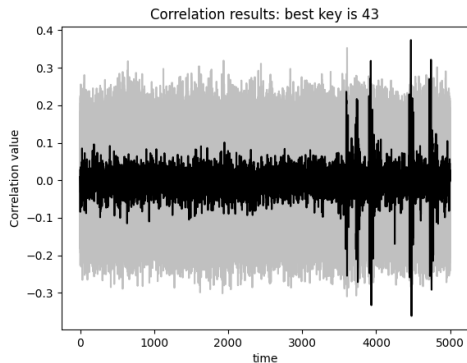
EXERCISE 1, LAB SHEET 2, CONT.

What happens when targeting AddRoundKey?



EXERCISE 1, LAB SHEET 2, CONT.

What happens when changing to the LSB power model?



The attacks deliver very similar results: no wonder, both statistical methods use the same information. There is no magic in the use of statistics ... (one can prove that when given the same model, both methods are asymptotically equivalent).

NUMBER OF NEEDED TRACES, SNR

We judge/compare attack outcomes via the number of traces that are needed to recover a subkey.

(Unsurprisingly) we find that traces with more data dependency (aka signal) and less irrelevant contributions (aka noise), lead to better attacks.

Also unsurprisingly, for “simple” attacks (like differential input attacks with “simple” leakage prediction functions, and distinguishers), we can make an explicit link between the number of needed traces, the signal-to-noise ratio, and a distinguisher outcome.

$$SNR(t) = \frac{Var(P_{exp}(t))}{Var(P_{noise}(t))} \quad (2)$$

ROADMAP

Side Channel Attacks

Attacks

Number of Needed Traces, SNR

Countermeasures

Masking, aka (d, d) Boolean Secret Sharing

NUMBER OF NEEDED TRACES, SNR

If we assume that incorrect key guesses will converge to 0 correlation, and that the correct key guess eventually converges to correlation = ± 1 , we get the “rule of thumb” for correlation based attacks:

$$n \approx \frac{1}{\rho^2}$$

I showed that we can rewrite the correlation estimator:

$$\begin{aligned} \rho(H_i, P_{total}) &= \\ \dots &= \frac{\rho(H_i, P_{exp})}{\sqrt{1 + \frac{1}{SNR}}} \end{aligned}$$

Now if we assume a really good adversary, i.e. $\rho(H_i, P_{exp})$ (the adversary has the perfect model), then we get the relationship:

$$\rho(H_i, P_{total}) \equiv \sqrt{SNR}.$$

NUMBER OF NEEDED TRACES, SNR

Having such relationships for “simple attacks” is useful for at least two reasons:

1. The rule of thumb enables to reason about the effectiveness of countermeasures: no magic in what happens when we increase the noise, or try and reduce the signal
2. Using the correlation estimate we can be rather precisely describe how different misalignment strategies will impact on the signal (which tells us via the rule of thumb something about the number of needed traces)
3. The rule of thumb gives a rough idea about how many traces one should expect upon seeing certain (simulated/tested) correlation values in attacks.

Disclaimer: there are more precise formulas available for estimating the number of needed traces.

EXERCISE 2, LAB SHEET 2

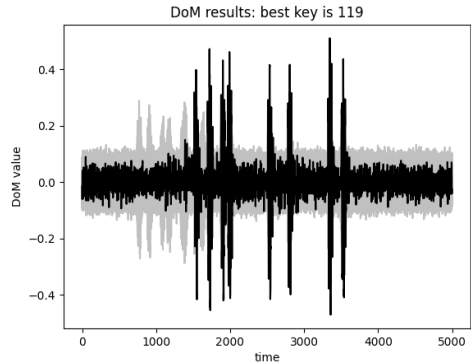
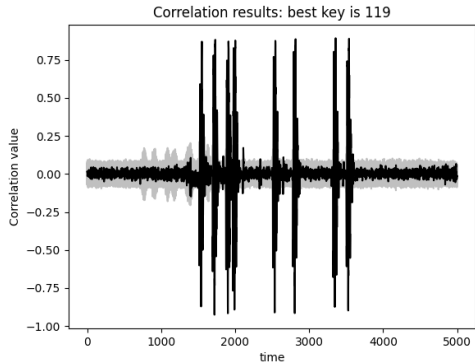
Given a(nother) dataset, produce “evolution” plots for correlation and DoM attacks and compare them with the rule of thumb.

Produce SNR plots for three different signal definitions, and try and make sense of them: why are there different peaks, do they align with attack outcomes?

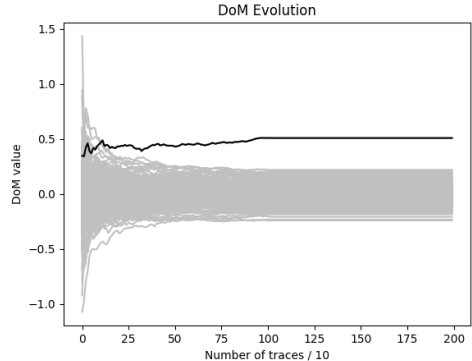
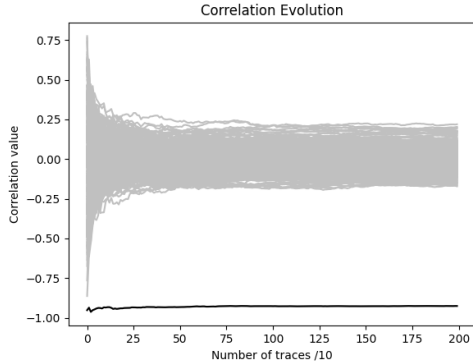
The dataset is very similar to WS1 but there are now 2000 traces to play with. In principle, this would enable repeat experiments, which would be necessary to get “good” estimates for number of needed traces, etc.

The first step here is to run attacks on the SubBytes output using a HW/LSB power model to determine the secret key byte: it should come out as 119. The correlation attack should show the highest (negative) peak at 1721. The DoM attack should have the highest peak at 3347.

EXERCISE 2, LAB SHEET 2, CONT.

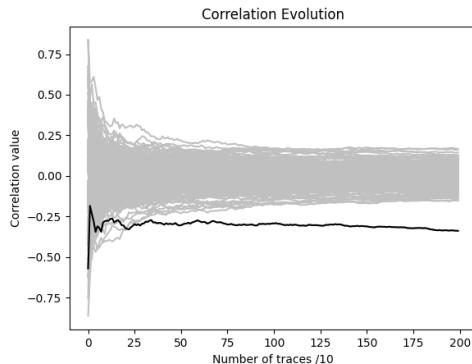
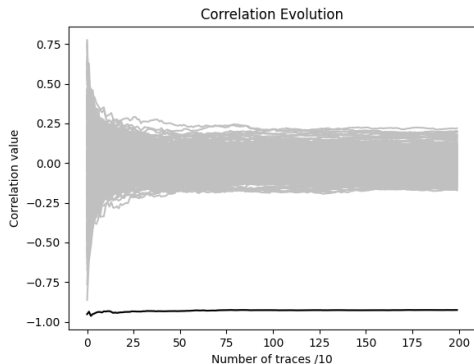


EXERCISE 2, LAB SHEET 2, CONT.



Around 10 traces suffice to succeed. For a correlation of 0.95, this is what we would expect according to the estimates in the table that is part of the notes. The same formula cannot be used for the DoM distinguisher.

EXERCISE 2, LAB SHEET 2, CONT.



Therefore we change compute an evolution plot for a single bit attack on SubBytes with the correlation distinguisher. It shows that the correlation score reaches approximately 0.3. According to the table, this implies that just under 300 traces should suffice, which corresponds to the evolution plot.

EXERCISE 2, LAB SHEET 2, CONT.

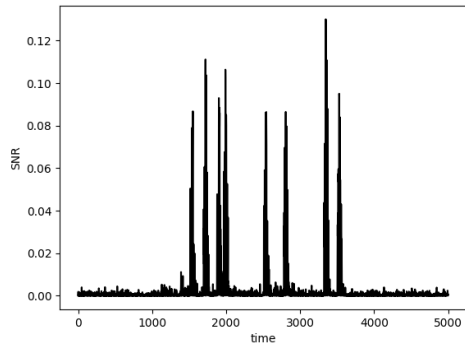
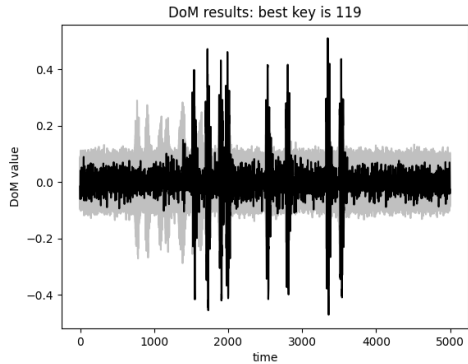
Now let's look at SNR plots. The SNR function in Python requires three inputs. The number of labels (i.e. the number of signal classes that we will divide our traces up into), the actual label for each trace, and then a set of traces.

All the intermediate values (aka targets) that we consider are 8 bit values.

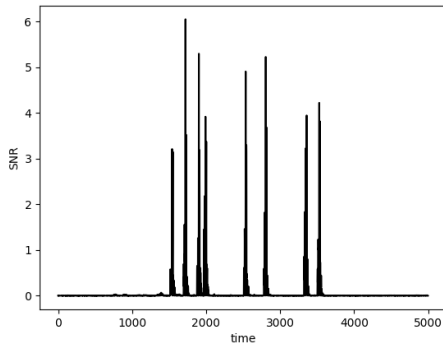
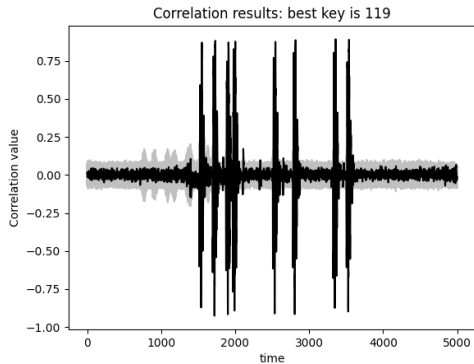
The exercise asks for three leakage models:

- ▶ The LSB model, aka a classification into two classes, which is required for DoM based attacks.
- ▶ The HW model. This implies that we have 9 classes.
- ▶ The “identity” model. This is something we did not discuss. It essentially means that we assume that the predictable leakage is due to just one intermediate value (the other models assume this too), and we make no further assumptions about how many signal classes there are.

EXERCISE 2, LAB SHEET 2, CONT.

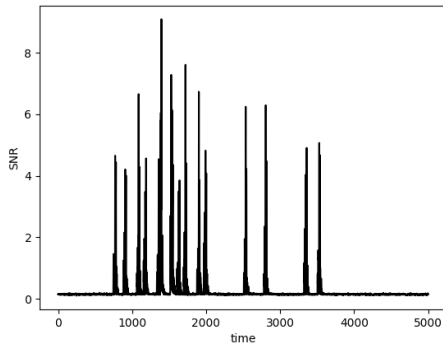


EXERCISE 2, LAB SHEET 2, CONT.



EXERCISE 2, LAB SHEET 2, CONT.

With the ID model, we find confirmation that there are more leaking trace points than what the HW and LSB models can exploit. This is clear evidence that more complex and multivariate models should lead to much better attacks. Indeed, upon training deep networks for (various) AES implementations on micro-controllers, we found that most of them (even with simple masking and hiding countermeasures) can be broken with a single trace.



ROADMAP

Side Channel Attacks

Attacks

Number of Needed Traces, SNR

Countermeasures

Masking, aka (d, d) Boolean Secret Sharing

THREE TYPES OF COUNTERMEASURES

An adversary aims to predict intermediate values and their leakage. Attacks benefit if information leakage is “aligned” across traces (i.e. the prediction happens within some “window” within a trace).

Hiding: we aim to change the when/where/how something leaks. I.e. the adversary can predict the actual values that are manipulated inside the device with just guessing parts of the key; but the targeted intermediate may occur at different time points, it may have very complex leakage (or leak very little), etc.

Masking: we aim to randomise how intermediate values are represented. I.e. the adversary can no longer predict the actual values that are manipulated inside the device by just guessing parts of the key; the targeted intermediate is now represented somewhat randomly and its representation changes in every encryption.

Protocol: we aim to limit the lifetime of a key so that an adversary cannot get enough observations to mount an attack.

THREE TYPES OF COUNTERMEASURES, CONT.

Protocol countermeasures only make sense if single-input attacks are very unlikely to succeed. Therefore, mostly we have to implement hiding/masking to get to that point.

Circumventing hiding typically requires to “additively combine” trace points. This leads to a linear increase in the number of needed traces. Masking typically requires to “multiplicatively combine” trace points. This leads, if all goes well, to an exponential increase in the number of needed traces.

THREE TYPES OF COUNTERMEASURES, CONT.

Protocol countermeasures only make sense if single-input attacks are very unlikely to succeed. Therefore, mostly we have to implement hiding/masking to get to that point.

Circumventing hiding typically requires to “additively combine” trace points. This leads to a linear increase in the number of needed traces.

Masking typically requires to “multiplicatively combine” trace points. This leads, if all goes well, to an exponential increase in the number of needed traces.

The promise of an “exponential increase” in the number of needed traces is the very reason why masking countermeasures are nearly always deployed in practice.

ATTACKING MASKING

“Mean-free product combining”: for two trace points x and y , compute a new points z with

$$z = (x - E(x))(y - E(y)).$$

The attack then works on a new trace that is based on points z , and it proceeds like the attacks that we studied before.

Caveat: must be carefull about what we predict.

Example simulation shows how an attack works when targetting the joint leakage of $(m, S(x) \oplus m)$. The simulation produces leakage on $m, S(x) \oplus m, n$ (n is just completely random).

ATTACKING MASKING, EXERCISE 3, LAB SHEET 2

The exercise asks you to play around with the simulation for masking. **1.** do an attack when leakage on the input and output of an Sbox are available and both use the same randomness.

So we should simulate leakage on $x \oplus m$, $S(x) \oplus m$ and see if we can attack it.

The simulation of this scenario is a simple extension to the existing code:

```
ark = AddRoundKey(inputs, key)
sb = SubBytes[ark]
sb_share= rng.integers(low=0, high=255, size=(num_inputs,1))
ark_share = np.concatenate((sb_share, np.bitwise_xor(sb_share,ark)),axis=1)
sb_share = np.concatenate((ark_share, np.bitwise_xor(sb_share,sb)),axis=1)
```

[Remember to take the additional point into account when doing the mean-free product combining.]

ATTACKING MASKING, EXERCISE 3, LAB SHEET 2, CONT.

If you simply run the PCor script now on the generated traces, your attack will not succeed. But why?

The pre-processing combines trace points; the prediction must consider what “corresponding intermediate variables” are combined because of it.

The simulation was set up to deal with the joint leakage of $(m, S(x) \oplus m)$: the mean-free product combining “creates” a traces point that leaks proportional to $HW(m \oplus S(x) \oplus m) = S(x)$.

I asked you to exploit the joint leakage of $(m \oplus x, S(x) \oplus m)$. Via inference you should realise that this implies that we need to predict $HW(m \oplus x \oplus S(x) \oplus m) = HW(x \oplus S(x))!$

So in PCor() you want to predict like this:

```
pred_leak = HW[np.bitwise_xor(sb,ark) ]
```

ATTACKING MASKING, EXERCISE 3, LAB SHEET 2, CONT.

3. do an attack when the input and output masks to SubBytes are different, and the masks are leaking.

Simulation is straightforward, and the attack really proceeds in the way as originally implemented in the simulation.

The prediction for the SubBytes input when targetting $(m, x \oplus m)$ is obviously x .

The prediction for the SubBytes output is exactly as in the simulation that I gave you.

ATTACKING MASKING, EXERCISE 3, LAB SHEET 2, CONT.

3. do an attack when the input and output masks to SubBytes are different, and the masks are not leaking.

Simulation is straightforward, but the attack is not. We cannot target the joint leakage of input and output because they are not sharing the same mask.

The text says that the same input and output masks are however used for all state bytes.

Therefore an attack now needs to target two SubBytes operations (either input or output will work, and we know from previous experience that output is easier). So the simulation now needs to sample another key byte and leak, using the same randomness on the masked SubBytes output of the other key byte too.

The prediction is the $HW(S(x) \oplus S(y))$, which requires the simultaneous guessing of 2 key bytes (i.e. we now have 256^2 values to test).

ROADMAP

Side Channel Attacks

Attacks

Number of Needed Traces, SNR

Countermeasures

Masking, aka (d, d) Boolean Secret Sharing

A (d, d) BOOLEAN SECRET SHARING SCHEME

We assume that the scheme is to work on bit strings (aka finite field elements of F_{2^l}).

```
1  def Bdd.Share(v, d) :  
2      for i = 1 upto d - 1  
3           $v_i \xleftarrow{\$} \{0, 1\}^l$   
4       $v_d = v_1 \oplus v_2 \dots v_{d-1} \oplus v$   
5      return  $\{v_1, v_2, \dots, v_d\}$ 
```

Listing: Boolean (d, d) secret sharing: Share function

```
1  def Bdd.Reconstruct( $\{v_1, v_2, \dots, v_d\}$ ) :  
2      return  $(v_1 \oplus v_2 \oplus \dots \oplus v_d)$ 
```

Listing: Boolean (d, d) secret sharing: Reconstruct function

USE TO SECURE CRYPTO IMPLEMENTATIONS (E.G. AES)

We apply the *Bdd.Share()* function at the start of an encryption to every byte of the state and the key state.

After the initial sharing out of all state/key bytes, we need to perform AES in an “MPC” style fashion.

We apply the *Bdd.Reconstruct()* function at the end of an encryption to every state byte.

Utilising fresh randomness for all state and key bytes may be practically challenging: but, to guarantee security up to “order d ”, we must ensure that no $d - 1$ intermediate variables use the same randomness for their shares.

E.g. consider $(v_1, v_2, v_1 \oplus v_2 \oplus a)$ and $(v_1, v_2, v_1 \oplus v_2 \oplus b)$. An adversary can succeed with a differential input attack recombining just two shares (it could recover information about the joint distribution (a, b)).

EXAMPLE: ADDROUNDKEY, MIXCOLUMNS

AddRoundKey: $\text{ARK}(v, k) = v \oplus k$ (for all bytes). When we work with shares, we are applying ARK to all pairs of state and key shares. Suppose $v = (v_1, v_2, \dots, v_d)$ and $k = (k_1, k_2, \dots, k_d)$, then $\text{ARK}(v_i, k_i) = v_i \oplus k_i$. This is correct because we can get $\text{ARK}(v, k) = v \oplus k$ via $\text{BDD.Reconstruct}(\{v_1 \oplus k_1, v_2 \oplus k_2, \dots, v_d \oplus k_d\})$.

MixColumns (MC): Consider a $(2, 2)$ scheme where $v_i = (m_i, v_{m_i}) = (m_i, v_i \oplus m_i)$, then because of the linearity of MixColumns we have that:

$$\text{MC}(v_{m_1}, v_{m_2}, v_{m_3}, v_{m_4}) = \text{MC}(v_1, v_2, v_3, v_4) \oplus \text{MC}(m_1, m_2, m_3, m_4).$$

And therefore

$$\text{BDD.Reconstruct}(\text{MC}(m_1, m_2, m_3, m_4), \text{MC}(v_{m_1}, v_{m_2}, v_{m_3}, v_{m_4})) = \text{MC}(v_1, v_2, v_3, v_4).$$

The same property holds for a (d, d) Boolean secret sharing scheme.

SECURITY ARGUMENT: PROBING SECURITY FOR d SHARES

Values in red colour are secret. There is a long term secret key sk . The implementation represents all intermediate values v_j via a (d, d) Boolean secret sharing scheme, i.e. as shares $v_{j,i}$.

An adversary who gets $d - 1$ intermediate variables (aka shares), cannot infer any information about the corresponding state byte(s)/ key byte(s).

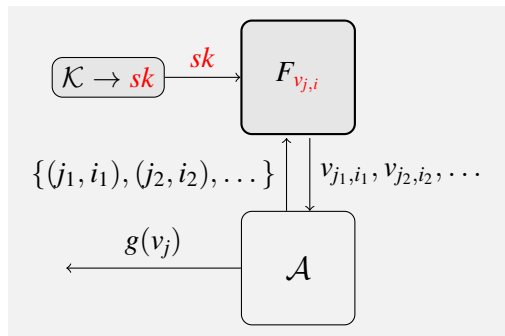


Figure: Security game: resilience against adversary who can request d shares from an implementation. The adversary wins if they learn some function about an intermediate values $g(v_j)$.

HOW DOES THAT RELATE TO SIDE-CHANNEL LEAKAGE?

Adversary requests shares.

With $d - 1$ shares the adversary should learn no information about the resp. intermediate values.

With d shares the scheme enables full reconstruction.

Adversary uses points from leakage traces.

Combining $d - 1$ points from each trace should provably not enable an attack (no matter how many traces are available).

Combining d points should provably increase the effort by an exponential factor of d .

By using a “provably secure masking scheme” we hope for implementations where we can provably guarantee that the adversary always must combine d trace points to succeed.

SECRET SHARING SUBBYTES WHEN $d = 2$

SubBytes is a round function that applies a function $S()$ to each byte of the states. It is non-linear with respect to the Boolean addition: $S(v \oplus m) \neq S(v) \oplus S(m)$.

The function $S()$ is often implemented as a table look-up. Therefore we have an easy work-around if $d = 2$. Rather than work with S we are going to compute a table $S_m()$ that takes into account the input sharing $(m, v \oplus m)$. So $S_m(v \oplus m) = S(v) \oplus m$, and therefore $\text{BDD.Reconstruct}(m, S(v) \oplus m) = S(v)$.

```
1 def compute_Sm(table S, byte m):  
2     for i = 0 to 255 do:  
3          $S_m(i) = S(i \oplus m) \oplus m$   
4     return  $S_m$ 
```

Listing: Computing $S_m()$

Notice that if we use fresh randomness for the shares of the different state bytes, then we would need to compute such a table specific to each state byte.

SUBBYTES AS FUNCTION OVER $\mathbb{Z}_{2^8}$

SubBytes is defined as the function $S(x) = A \cdot x^{-1} + b$. Here, A is a matrix with elements in $\mathbb{Z}_2$ and b is a vector with elements in $\mathbb{Z}_2$.

Hence the problem with S stems from the fact that it is based on a finite field inversion (which is a multiplicative function and not additive)¹.

Multiplicative: recall that $a^{-1} \equiv a^{p-2} \pmod{p}$; so $a^{-1} = a^{254}$ over $\mathbb{Z}_{2^8}$.

We can do an exponentiation as a sequence of squares and multiplies.

Which brings us to the question: how can we do a “probing secure (finite field) multiplication” when a is represented via a (d, d) Boolean masking scheme?

¹This relates to your summative coursework

SECRET SHARED MULTIPLICATION FOR $d = 2$ (INSECURE)

The two secret inputs to the multiplication are represented as $a = (a_1, a_2) = (a_1, a \oplus a_1)$ and $b = (b_1, b_2) = (b_1, b \oplus b_1)$.

Value		
a	a_1	a_2
b	b_1	b_2
$c = a \cdot b$	$a_1 \cdot b_1$	$a_2 \cdot b_2 \oplus a_1 \cdot b_2 \oplus a_2 \cdot b_1$

The value c correctly gives the product of a and b . But how about security in our probing game?

We must write out the intermediate variables that an implementation would actually produce.

EXEMPLARY “PROGRAM FLOW”

$$v_1 = a_1 \cdot b_1$$

$$v_2 = a_1 \cdot b_2$$

$$v_3 = a_2 \cdot b_1$$

$$v_4 = a_2 \cdot b_2$$

$$v_5 = v_4 \oplus v_2$$

$$v_6 = v_5 \oplus v_3$$

$$c_1 = v_1$$

$$c_2 = v_6$$

I write the “envisioned steps” so that the result is each assigned its own, new variable:

“Static single assignment” (SSA) form.

The adversary thus just sends a label for the index as part of the game, and then gets back one of these intermediate variables.

The first four intermediate variables are, per construction, independent of a and b , because they are computed using only one share of a and b . The intermediate values v_5 and v_6 are both problematic. We can see this in two different ways. One option is to notice that if $a = 0$ then $v_5 = (a_2 \oplus a_1) \cdot b_2 = a \cdot b_2 = 0$ (no matter what the random values a_1, b_2 are). Therefore the adversary can easily distinguish RR and LR worlds (by supplying $a = 0$ as input).

FIXING THE PROBLEM

$$v_1 = a_1 \cdot b_1$$

$$v_2 = a_1 \cdot b_2$$

$$v_3 = a_2 \cdot b_1$$

$$v_4 = a_2 \cdot b_2$$

$$v_5 = z \oplus v_4$$

$$v_6 = v_5 \oplus v_2$$

$$v_7 = v_6 \oplus v_3$$

$$c_1 = v_1 \oplus z$$

$$c_2 = v_7$$

We add another “one-time encryption layer” to the computation of v_5 .

We do this by adding in a fresh random value z before adding on further terms.

To ensure correctness, we “remove” the fresh random z by also adding it into the first output share.

In the notes, I rewrite the Real and Random worlds, to represent this computation. In the Random world, we can now simply return a random value if the adversary asks for v_5, v_6, v_7 . So we can do a perfect simulation.

ISW MULTIPLICATION FOR $d = 2$, CONT.

We utilise the space to introduce fresh randomness, $r_{1,2}$. We place it into the empty space and “flip” it as well so that we get a correct result.

	b_1	b_2	
a_1	a_1b_1	$r_{1,2}$	$c_1 = b_1a_1 \oplus r_{1,2}$
a_2	$(r_{1,2} \oplus a_1b_2) \oplus a_2b_1$	a_2b_2	$c_2 = a_2b_2 \oplus ((r_{1,2} \oplus a_1b_2) \oplus a_2b_1)$

Notice that I have “grouped” certain computations by placing brackets around them.

This indicates that there is an order to the computations, and you can check that it exactly matches what happens in SecMult.

ISW for $d = 2$ is what we found also on the previous slide.

SECURITY ARGUMENT (SPECIFIC FOR $d = 2$)

The security argument is exactly as before, but the proof that we went through in the lectures is of such a nature that it holds for any d .

- ▶ Intermediate variables “on or above the diagonal” can be simulated, and we “remember” that we can compute them by storing their indices in two lists: $a_1b_1, a_2b_2, r_{1,2}$ all only depend on at most one share of any secret. Therefore we can randomly sample this share and we store the indices for a in I and for b in J .
- ▶ Variables “below the diagonal” depend on values that are also above the diagonal. For instance if $r_{2,1}$ is requested then this is actually $r_{1,2} \oplus a_1b_2$. So if the adversary can request multiple intermediate variables, we have make sure that such dependency are faithfully simulated. We then “remember” what indices we need for this by storing the indices of a and b in both I and J .

We can complete the proof by observing that both lists can have at most length $d - 1$: we can write a simulator that never needs access to all shares of an of the secret inputs. This proof strategy is pretty sophisticated and most approaches that improve on ISW (in terms of efficiency) copy this argument.

EXERCISE 4, LAB SHEET 2

Consider another alternative way of summing up the partial products for a $d = 2$ masking scheme for a secure multiplication:

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_2b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_1b_2$

1. Show, theoretically, using any of the strategies that we used to reason about the security of secret shared computations, that this computation will leak information about either a or b or both.

EXERCISE 4, LAB SHEET 2, CONT.

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_2b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_1b_2$

It should be clear that an adversary who asks for any of the $a_i b_j$ will not succeed.

But an adversary who asks for a c_i should succeed. It may not be so obvious (in comparison to the lecture example) why. Let us consider c_1 :

$$\begin{aligned}c_1 &= a_1b_1 \oplus a_2b_2 = a_1b_1 \oplus (a \oplus a_1)(b \oplus b_1) \\&= a_1b_1 \oplus ab \oplus a_1b \oplus ab_1 \oplus a_1b_1 \\&= ab \oplus a_1b \oplus ab_1 \\&= ab_1 \oplus a_2b\end{aligned}$$

If we choose $a = b = 0$ then $c_1 = 0$ no matter what the random values a_1, b_1 are.

EXERCISE 4, LAB SHEET 2, CONT.

Consider another alternative way of summing up the partial products for a $d = 2$ masking scheme for a secure multiplication:

	b_1	b_2	
a_1	a_1b_1	a_1b_2	$c_1 = a_1b_1 \oplus a_2b_2$
a_2	a_2b_1	a_2b_2	$c_2 = a_2b_1 \oplus a_1b_2$

Consider c_2 :

$$\begin{aligned}c_2 &= a_2b_1 \oplus a_1b_2 = (a \oplus a_1)b_1 \oplus a_1(b \oplus b_1) \\&= ab_1 \oplus a_1b_1 \oplus a_1b \oplus a_1b_1 \\&= ab_1 \oplus a_1b\end{aligned}$$

If we choose $a = b = 0$ then $c_2 = 0$ no matter what the random values a_1, b_1 are. (This was kind of obvious from the previous slide).

EXERCISE 4, LAB SHEET 2, CONT.

As for an attack:

- ▶ The simulation works exactly like demonstrated in the scripts given to you.
- ▶ For the finite field multiplication the Galois package helps: $GF = \text{galois}.GF(2^{*}8)$, and then use this to generate convenient table look-ups for the multiplication.
- ▶ For the attack, one best uses a leakage model that distinguishes between zero and non-zero values: the inputs being equal to zero or not. So set $a = p \oplus k$ and $b = p \oplus k$, with known plaintext byte p and unknown key byte k . Then secret share a and b with independent randomness a_1, b_1 , and simulate traces with provided and varying p and some fixed k . Recover k by a conventional differential input attack, using the “zero value” model.

SIMULATION BASED SECURITY ARGUMENT

You must argue about the **distribution** of the output.

Just because a piece of code depends on a secret value, does not mean it leaks on it.

E.g. the intermediate value $x = (a \oplus a_1) \cdot b_2$ (with a_1, b_2 uniformly, independently chosen of a) depends on a but it does not leak on a .

This is because the distribution of x is the same no matter how we choose a : if a_1 is chosen independently of a from a uniform distribution then $a \oplus a_1$ is like the one-time pad, so it is distributed uniformly. The product of $a \oplus a_1$ and b_2 is then again independent of a (the product of two distributions that are independent of a is still independent of a).

EXERCISE 5, LAB SHEET 2

This is a “prototype exam question”: you are given (sufficient) details about some implementation of some algorithm (typically an encryption scheme, mostly a block cipher). You are then asked to consider questions relating to how to implement this algorithm securely on some platform (typically embedded, so you should always consider that an adversary has physical access).

1. Five steps: bookwork, see before in this slide set.
2. What side channels? Because it is an embedded system, the adversary is likely to get physically close, or even just have it for some time. So anything that benefits from close proximity is a particular threat: power/EM, timing.
3. It is a bit-oriented cipher and we are considering a hardware implementation. So Boolean secret sharing on a bit-level seems most sensible.

EXERCISE 5, LAB SHEET 2, CONT.

4. Bit-wise permutations in hardware are typically done by just “running wires” between state elements. So it is unlikely that anything within an S-box can be done, but potentially S-boxes could be processed in different orders by placing additional hardware elements that selects which one is done (assuming that they are not done in parallel); if they happen in parallel then no re-ordering is possible but perhaps adding more S-boxes that just work on random values and add noise could help. Dummy rounds could also be an option.

5.A. Not correct, a $d = 2$ scheme applied to all state elements, suffices assuming that an implementation takes care that randomness does not accidentally “cancels out”. One has to carefully assess though how, in the case of DES, the re-use of randomness accross all bits (which are the state elements) works out, and if necessary, introduce fresh randomness where appropriate. A $d = 3$ scheme should give provable security against an adversary who can combine trace points.

EXERCISE 5, LAB SHEET 2, CONT.

5.B. 3-DES means running DES three times with either two or three different keys: so the key length increases, but this has little to no impact on a side channel attack, which works on a divide-and-conquer principle. So no, a factor 3 increase is not correct.

5.C. Running DES with more rounds will not help much if it helps at all: more rounds could for instance be implemented by storing the result after 16 rounds already in an output register, and then keeping running DES. Perhaps the fact that the result is “transferred” after 16 rounds is visible in an EM/power trace, so the adversary knows when DES finished internally and they even get a better way of aligning power traces. In any case, the additional rounds will just leak at a different time, so when attacking e.g. the first round, they would have no impact at all on the SNR. Additional rounds that are dispatched at the beginning only will not have any effect on the last round. So dummy rounds would need to be randomly inserted in a way that is not visible in a side channel trace.